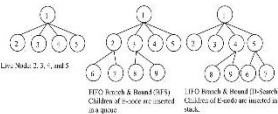


1. Define Algorithm Efficiency Algorithm efficiency refers to how well an algorithm performs in terms of time and space usage. It measures:>How fast an algorithm runs (time efficiency) How much memory it uses (space efficiency) 2. What is Time Complexity? Time complexity is the measure of the amount of time an algorithm takes to run as a function of input size n. //It is expressed using asymptotic notation like:$O(n)$, $O(n^2)$, $O(\log n)$ 3. What is Space Complexity? : Space complexity is the measure of the total memory required by an algorithm during execution. It includes:>Input space /// >Auxiliary space (extra memory used) 4. Define Big-O Notation with Example Big-O notation represents the upper bound (worst-case) time complexity of an algorithm. It shows how the running time grows with input size. // Example: // If an algorithm takes: $T(n) \approx 3n^2 + 2n + 1$ // Then: $O(n^2)$ 5. Define Omega (Ω) Notation. // Omega (Ω) notation represents the lower bound (best-case) time complexity of an algorithm. //It shows the minimum time required. 7. What is Asymptotic Analysis? : Asymptotic analysis is the method of evaluating the performance of an algorithm in terms of input size n, especially for large values of n. It focuses on: //Growth rate of time/space // Ignores constants and lower-order terms /// Expressed using: // Big-O, Omega, Theta 6. Define Theta (Θ) Notation. // Theta (Θ) notation represents the tight bound of an algorithm. It gives both: Upper bound // Lower bound // So it shows the exact growth rate. 8. What is Growth of Functions? // Growth of functions refers to how the running time of an algorithm increases as the input size increases. // It helps in comparing algorithms. Example: $O(1) < O(\log n) < O(n) < O(n^2) < O(2^n)$ Faster growth $\Rightarrow$ less efficient	1. What is Divide and Conquer? Divide and Conquer is an algorithm design technique in which a problem is: // Divided into smaller subproblems // Solved recursively // Combined to get the final solution Examples: // Merge Sort // Quick Sort 2. Define Binary Search. /// Binary search is a searching technique used to find an element in a sorted array by repeatedly dividing the search space into half. // Steps: // Find middle element // Compare with key // Search left or right accordingly 3. What is the Time Complexity of Binary Search? The time complexity of binary search is: Best Case: $O(1)$ // Worst Case: $O(\log n)$ Because the search space is halved at each step 4. Define Merge Sort : Merge sort is a divide and conquer sorting algorithm that: // Divides the array into halves // Sorts each half recursively // Merges the sorted halves It always produces a sorted array 5. What is the Time Complexity of Merge Sort? The time complexity of merge sort is: Best Case: $O(n \log n)$ // Worst Case: $O(n \log n)$ // Average Case: $O(n \log n)$ Because: // Division takes $\log n$ steps // Merging takes $O(n)$ time 6. Define Quick Sort : // Quick sort is a divide and conquer sorting algorithm that works by: // Selecting a pivot element // Partitioning the array into: // Elements less than pivot // Elements greater than pivot // Recursively sorting both parts It is an in-place sorting algorithm 10. What is Strassen Matrix Multiplication? Strassen's matrix multiplication is an efficient algorithm to multiply matrices using divide and conquer. It reduces number of multiplications from 8 to 7 Time Complexity: $O(n^{2.81})$ (faster than normal $O(n^3)$)	1. What is Greedy Algorithm? // A greedy algorithm is an approach that builds a solution step by step, choosing the best option at each step without considering future consequences. It always makes a locally optimal choice. Examples: // Kruskal's Algorithm // Dijkstra's Algorithm 2. What are Properties of Greedy Method? The greedy method has two main properties: a. Greedy Choice Property: A globally optimal solution can be obtained by choosing locally optimal choices. b. Optimal Substructure: The problem can be broken into subproblems, and optimal solution can be built from their solutions. 3. Define Minimum Spanning Tree (MST) A Minimum Spanning Tree (MST) is a spanning tree of a graph that: Connects all vertices // Has minimum total edge weight // Contains no cycles Used in network design, routing, etc. 4. What is Kruskal's Algorithm? Kruskal's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST). Steps: > Sort all edges in increasing order of weight /// >> Select smallest edge Add it if it does not form a cycle // Repeat until MST is formed Uses Disjoint Set (Union-Find) 5. What is Prim's Algorithm? Prim's algorithm is a greedy algorithm used to find the Minimum Spanning Tree (MST) of a graph. It starts from any vertex and: > Selects the minimum weight edge connected to the tree >Adds a new vertex to the tree // > Repeats until all vertices are included It always grows the MST one vertex at a time																																				
1. Define Dynamic Programming Dynamic Programming (DP) is a technique used to solve complex problems by breaking them into smaller subproblems, solving each subproblem once, and storing their results for reuse. It avoids repeated computations and improves efficiency. 2. What are Principles of Dynamic Programming? Dynamic Programming is based on two main principles: 1. Optimal Substructure: The optimal solution of a problem can be obtained from optimal solutions of its subproblems. 2. Overlapping Subproblems: The same subproblems are solved multiple times. DP stores their results to avoid recomputation. 3. Difference between Greedy and Dynamic Programming <table border="1"> <thead> <tr> <th>Feature</th><th>Greedy Method</th><th>Dynamic Programming</th></tr> </thead> <tbody> <tr> <td>Approach</td><td>Makes local optimal choice</td><td>Solves all subproblems</td></tr> <tr> <td>Decision</td><td>Does not reconsider choices</td><td>Considers all possibilities</td></tr> <tr> <td>Optimal Solution</td><td>Not always guaranteed</td><td>Always gives optimal solution</td></tr> <tr> <td>Complexity</td><td>Faster</td><td>Slower than greedy</td></tr> <tr> <td>Example</td><td>Kruskal, Dijkstra</td><td>LCS, Knapsack</td></tr> </tbody> </table>	Feature	Greedy Method	Dynamic Programming	Approach	Makes local optimal choice	Solves all subproblems	Decision	Does not reconsider choices	Considers all possibilities	Optimal Solution	Not always guaranteed	Always gives optimal solution	Complexity	Faster	Slower than greedy	Example	Kruskal, Dijkstra	LCS, Knapsack	🔥 9. What is Matrix Chain Multiplication? Matrix Chain Multiplication is a problem of determining the most efficient way to multiply a sequence of matrices. Goal: Minimize number of scalar multiplications It does not change result, only changes order of multiplication Solved using Dynamic Programming 🔥 10. What is Travelling Salesman Problem (TSP)? The Travelling Salesman Problem (TSP) is a problem where: A salesman must visit all cities exactly once // Return to the starting city Goal: Find the minimum cost path It is solved using: Dynamic Programming // Branch and Bound 6. Difference between Greedy and Branch & Bound <table border="1"> <thead> <tr> <th>Feature</th><th>Greedy Method</th><th>Branch and Bound</th></tr> </thead> <tbody> <tr> <td>Approach</td><td>Local optimal choice</td><td>Systematic search</td></tr> <tr> <td>Decision</td><td>Does not reconsider</td><td>Explores multiple options</td></tr> <tr> <td>Optimal Solution</td><td>Not always guaranteed</td><td>Always optimal</td></tr> <tr> <td>Complexity</td><td>Faster</td><td>Slower</td></tr> <tr> <td>Example</td><td>Kruskal, Dijkstra</td><td>Knapsack (B&B)</td></tr> </tbody> </table>	Feature	Greedy Method	Branch and Bound	Approach	Local optimal choice	Systematic search	Decision	Does not reconsider	Explores multiple options	Optimal Solution	Not always guaranteed	Always optimal	Complexity	Faster	Slower	Example	Kruskal, Dijkstra	Knapsack (B&B)	1. Define P Class P (Polynomial Time) class is the set of problems that can be solved in polynomial time by a deterministic algorithm. 👉 Time complexity: $O(n)$, $O(n^2)$, $O(n^3)$, etc. These problems are considered efficiently solvable. 2. Define NP Class NP (Non-deterministic Polynomial Time) class is the set of problems whose solutions can be verified in polynomial time. Not necessary that they are solved quickly, but: // Solution can be checked quickly Example: Travelling Salesman Problem 3. What is NP-Hard Problem? An NP-hard problem is a problem that is at least as hard as the hardest problems in NP. It may or may not belong to NP // No known polynomial-time solution Example: Travelling Salesman Problem (optimization version) 4. What is NP-Complete Problem? An NP-Complete problem is a problem that: 1. Belongs to NP class (solution can be verified in polynomial time) 2. Is also NP-hard (as difficult as the hardest problems in NP) In simple terms: It is both NP and NP-hard Example: Subset Sum Problem // Vertex Cover Problem
Feature	Greedy Method	Dynamic Programming																																				
Approach	Makes local optimal choice	Solves all subproblems																																				
Decision	Does not reconsider choices	Considers all possibilities																																				
Optimal Solution	Not always guaranteed	Always gives optimal solution																																				
Complexity	Faster	Slower than greedy																																				
Example	Kruskal, Dijkstra	LCS, Knapsack																																				
Feature	Greedy Method	Branch and Bound																																				
Approach	Local optimal choice	Systematic search																																				
Decision	Does not reconsider	Explores multiple options																																				
Optimal Solution	Not always guaranteed	Always optimal																																				
Complexity	Faster	Slower																																				
Example	Kruskal, Dijkstra	Knapsack (B&B)																																				
Basics of Randomized Algorithms A randomized algorithm is an algorithm that uses random numbers to make decisions during execution. Output or running time may vary for the same input Key Features : Uses random choices // Performance measured in expected time // Useful for complex problems Types of Randomized Algorithms 1. Las Vegas Algorithm: Always gives correct result // Running time is random Example: Randomized Quick Sort ♦ 2. Monte Carlo Algorithm : Running time is fixed // May give incorrect result with small probability Advantages: ✓ Simple and fast // ✓ Avoids worst-case scenarios // ✓ Good average performance Example: Randomized Quick Sort: Pivot is chosen randomly Average Time: $O(n \log n)$ Explain String Matching Algorithms String matching is the process of finding a pattern (P) in a text (T). Goal: Find all positions where pattern occurs in text Types of String Matching Algorithms 1. Naive (Brute Force) Algorithm Compare pattern with text at every position Steps: // Align pattern with text // Compare characters // If mismatch $\rightarrow$ shift pattern Time Complexity: $O(n \times m)$ 2. KMP (Knuth-Morris-Pratt) Algorithm : Avoids repeated comparisons using LPS array LPS: Longest Prefix = Suffix // Time Complexity: $O(n + m)$ 3. Rabin-Karp Algorithm: Uses hashing technique Steps: // Compute hash of pattern // Compare with substring hash // If match $\rightarrow$ verify Time Complexity: Average: $O(n + m)$ // Worst: $O(n \times m)$	Explain Circuit Satisfiability (SAT) Circuit Satisfiability (SAT) is the problem of determining whether there exists an assignment of input values (TRUE/FALSE) to a Boolean circuit such that the output is TRUE. Explanation : A Boolean circuit consists of: // Input variables ($x_1, x_2, \dots$) // Logic gates (AND, OR, NOT) The task is: Check if there is any combination of inputs that makes output = 1 (TRUE) Example Circuit: Output = $(x_1 \text{ AND } x_2) \text{ OR } x_3$ Try values: // $x_1 = 1, x_2 = 1, x_3 = 0$ // Output = $(1 \text{ AND } 1) \text{ OR } 0 = 1$ // So circuit is satisfiable Important Points : SAT was the first NP-Complete problem Used in: // Logic design // Verification // AI What is Approximation Algorithm? An Approximation Algorithm is an algorithm used to find a near-optimal solution for optimization problems, especially when exact solution is difficult (NP-Hard problems). Key Idea Exact solution is costly // So we find good enough solution quickly Approximation Ratio Measures quality of solution: For minimization: Approx solution $\leq r \times$ Optimal For maximization: // Approx solution $\geq$ Optimal / r Example: Vertex Cover (2-approximation): // Select both endpoints of an edge // Gives solution within 2 $\times$ optimal Advantages Fast and efficient // Works for NP-Hard problems // Provides guaranteed bound Disadvantages Not exact solution // Accuracy depends on algorithm	P, NP, NP-Hard, NP-Complete 1. P (Polynomial Time) Class P class consists of problems that can be solved in polynomial time by a deterministic algorithm. 👉 Time complexity: $O(n)$, $O(n^2)$, $O(n^3)$, etc. These problems are considered efficient and easy to solve 2. NP (Non-deterministic Polynomial Time) Class NP class consists of problems whose solutions can be verified in polynomial time. Not necessary that they are solved quickly, but: Given a solution $\rightarrow$ can be checked quickly Example: Travelling Salesman Problem 3. NP-Hard Problems A problem is NP-Hard if it is at least as hard as the hardest problems in NP. Key points: May or may not belong to NP No known polynomial-time solution Example: // TSP (optimization version) 4. NP-Complete Problems A problem is NP-Complete if: // It belongs to NP // It is also NP-Hard These are the hardest problems in NP Examples: // Subset Sum // Vertex Cover // SAT Relationship (IMPORTANT) $P \subseteq NP$ // $NP\text{-Complete} \subseteq NP$ $NP\text{-Complete} \subseteq NP\text{-Hard}$																																				

6. Define Dijkstra's Algorithm Dijkstra's algorithm is used to find the shortest path from a source vertex to all other vertices in a weighted graph (with non-negative weights). // Steps: > Initialize distance of source = 0 // >> Select vertex with minimum distance // >>>Update distances of adjacent vertices // >>>Repeat until all vertices are visited 7. What is Bellman-Ford Algorithm? Bellman-Ford algorithm is used to find the shortest path from a source vertex to all other vertices. // It works even with negative edge weights. Steps: >>Initialize distances // >>>Relax all edges (V-1) times // >>>Check for negative cycles 8. Define Huffman Coding // Huffman coding is a greedy algorithm used for data compression. // It assigns: // Short codes to frequent characters // Long codes to less frequent characters It uses a binary tree (Huffman tree) 9. What is Activity Selection Problem? The Activity Selection Problem is a problem of selecting the maximum number of non-overlapping activities from a given set of activities. Each activity has: // Start time // Finish time Goal: // Select activities such that no two activities overlap Maximize number of activities Solved using Greedy method (earliest finish time first) 10. What is Fractional Knapsack Problem? The Fractional Knapsack Problem is a problem where items can be divided into fractions to maximize total profit. Each item has: // Weight // Value Goal: Maximize value within given capacity Strategy: Select items based on highest value/weight ratio It is solved using Greedy algorithm	5. Difference between NP-Hard and NP-Complete <table border="1"> <thead> <tr> <th>Feature</th><th>NP-Hard</th><th>NP-Complete</th></tr> </thead> <tbody> <tr> <td>Definition</td><td>At least as hard as NP problems</td><td>NP + NP-Hard</td></tr> <tr> <td>Belongs to NP</td><td>Not necessary</td><td>Yes</td></tr> <tr> <td>Solution verification</td><td>May not be possible in polynomial time</td><td>Possible in polynomial time</td></tr> <tr> <td>Examples</td><td>TSP (optimization)</td><td>Subset Sum, Vertex Cover</td></tr> </tbody> </table> 10. What is Approximation Algorithm? An approximation algorithm is an algorithm used to find a near-optimal solution for optimization problems where finding the exact solution is difficult. It provides: A solution close to optimal // In reasonable time Used for NP-hard problems 11. Define Randomized Algorithm A randomized algorithm is an algorithm that uses random numbers to make decisions during execution. It may produce:Different outputs for same input Expected good performance Types: // Las Vegas (always correct, random time) Monte Carlo (may give approximate result) 12. What is String Matching? String matching is the process of finding whether a pattern string exists in a given text string. Goal: Find occurrence of pattern in text Examples: Naive string matching // KMP algorithm	Feature	NP-Hard	NP-Complete	Definition	At least as hard as NP problems	NP + NP-Hard	Belongs to NP	Not necessary	Yes	Solution verification	May not be possible in polynomial time	Possible in polynomial time	Examples	TSP (optimization)	Subset Sum, Vertex Cover	9. Define Recurrence Relation. // A recurrence relation is an equation that defines a function in terms of its smaller input values. // Used to analyze recursive algorithms. Example: // $T(n) = 2T(n/2) + n$ // Solved using: Substitution method // Recursion tree // Master's Theorem 10. What is Master's Theorem? // Master's Theorem is a formula used to solve recurrence relations of the form: $T(n) = aT\left(\frac{n}{b}\right) + f(n)$ It provides a quick way to find time complexity. Cases: - If $f(n) = O(n^{\log_b a})$ $\rightarrow T(n) = \Theta(n^{\log_b a})$ - If $f(n) = \Theta(n^{\log_b a})$ $\rightarrow T(n) = \Theta(n^{\log_b a} \log n)$ - If $f(n) = \Omega(n^{\log_b a})$ $a f(n/b) \leq c f(n)$ $\rightarrow T(n) = \Theta(f(n))$ 7. What is the Worst Case of Quick Sort? The worst case occurs when the pivot divides the array into unequal parts, such as: // Already sorted array // Reverse sorted array // Time Complexity (Worst Case): $O(n^2)$ 8. Define Heap Sort // Heap sort is a sorting algorithm based on heap data structure (binary heap). // Steps: >Build a max heap // >>Extract maximum element // >>>Repeat until sorted It is an in-place and comparison-based sorting algorithm 9. What is Recurrence Relation in Divide and Conquer? In divide and conquer, a recurrence relation represents the time complexity of recursive algorithms. General form: // $T(n) = aT(n/b) + f(n)$ // Where: // a= number of subproblems // b= size division factor // $f(n)$= extra work
Feature	NP-Hard	NP-Complete															
Definition	At least as hard as NP problems	NP + NP-Hard															
Belongs to NP	Not necessary	Yes															
Solution verification	May not be possible in polynomial time	Possible in polynomial time															
Examples	TSP (optimization)	Subset Sum, Vertex Cover															
6. Define Polynomial Time Polynomial time refers to the time complexity of an algorithm that can be expressed as a polynomial function of input size n. General form: $O(n^k)$ // (where k is a constant) Examples: $O(n)$, $O(n^2)$, $O(n^3)$ These algorithms are considered efficient and practical 7. Define Non-Polynomial Time Non-polynomial time refers to the time complexity of an algorithm that cannot be expressed as a polynomial function of input size n. These algorithms take very high time for large inputs Examples: $O(2^n)$, $O(n!)$ Such problems are considered inefficient and hard to solve 8. What is Vertex Cover Problem? The Vertex Cover Problem is a problem in graph theory where: Given a graph, find a set of vertices such that: // Every edge of the graph is incident to at least one selected vertex Goal: Find the minimum number of vertices It is an NP-Complete problem 9. What is Subset Sum Problem? The Subset Sum Problem is a problem where: Given a set of numbers and a target value, // Find a subset whose sum equals the target Example: Set = {2, 4, 6}, Target = 6 // Subset = {2, 4} It is an NP-Complete problem	1. What is Branch and Bound? Branch and Bound is an algorithm design technique used to solve optimization problems. It works by: Branching $\rightarrow$ dividing problem into subproblems Bounding $\rightarrow$ calculating a bound to eliminate non-promising solutions It reduces search space and finds optimal solution 2. What is Bounding Function? A bounding function is used to calculate the best possible solution (upper or lower bound) for a node. It helps to: // Decide whether to explore or discard a node If bound is not promising $\rightarrow$ node is pruned (ignored) 3. Define LC Search LC (Least Cost) Search is a type of Branch and Bound method where the node with the least cost (minimum bound value) is selected first. It uses a priority queue // Always expands the most promising node 4. What is FIFO Branch and Bound? FIFO (First-In-First-Out) Branch and Bound is a method where nodes are explored in the order they are generated. It uses a queue (FIFO structure) // The node inserted first is expanded first // It is similar to Breadth-First Search (BFS) 5. Define 0/1 Knapsack Problem The 0/1 Knapsack Problem is an optimization problem where: // Each item has a weight and value // We either select (1) or reject (0) an item Goal: Maximize total value without exceeding capacity No fractional selection allowed	4. What is Optimal Substructure? Optimal substructure means that the optimal solution of a problem can be obtained from optimal solutions of its subproblems. If subproblems are solved optimally, the overall solution will also be optimal. Example: // Shortest path // Knapsack problem 5. What is Overlapping Subproblems? Overlapping subproblems means that the same subproblems are solved repeatedly during computation. Dynamic Programming stores results of subproblems to avoid recomputation. // Example: // Fibonacci series 6. Define Rod Cutting Problem The Rod Cutting Problem is a DP problem where: A rod of length n is given // Each piece has a price Goal: Cut the rod into pieces to maximize total profit It uses optimal substructure + overlapping subproblems 7. What is Floyd-Warshall Algorithm? Floyd-Warshall algorithm is a Dynamic Programming algorithm used to find the shortest paths between all pairs of vertices in a graph. It works for: // Directed graphs // Weighted graphs (can handle negative weights, but no negative cycles) It uses a matrix-based approach 8. Define LCS (Longest Common Subsequence) The Longest Common Subsequence (LCS) is the longest sequence that appears in the same order in two given sequences (not necessarily contiguous). Example: // For strings "ABCBDAB" and "BDCAB", LCS = "BCAB" // It is solved using Dynamic Programming															
📌 Definitions: - Branch and Bound is a state space search method in which all the children of a node are generated before expanding any of its children. Live-node: A node that has not been expanded. - It is similar to backtracking technique but uses BFS-like search.  - Dead-node: A node that has been expanded - Solution-node 📌 LC-Search (Least Cost Search): - The selection rule for the next E-node in FIFO or LIFO branch-and-bound is sometimes "blind", i.e. the selection LC (Least Cost) Search is a type of Branch and Bound technique in which the node with the minimum cost (or best bound value) is selected first for expansion. // It uses a priority queue // Also called Best-First Search Key Idea: Each node has a cost / bound value // Always expand the node with least cost (most promising) // Helps reach optimal solution faster Algorithm Steps : 1.Create root node and compute its cost 2.Insert root into priority queue // 3.Repeat: // Select node with minimum cost // If it is solution $\rightarrow$ stop // Else expand node // Compute cost of children // Insert promising nodes into queue	Explain Vertex Cover Problem The Vertex Cover Problem is a problem in graph theory where we select a set of vertices such that every edge of the graph is connected to at least one selected vertex. Problem Statement Given a graph $G = (V, E)$, find a subset $C \subseteq V$ such that: For every edge $(u, v) \in E$, // at least one of u or v is in C Goal: Find the minimum size vertex cover Example Graph edges: // (1,2), (2,3), (3,4) One vertex cover: // {2, 3} Covers all edges: // (1,2) $\rightarrow$ 2 // (2,3) $\rightarrow$ 2 or 3 // (3,4) $\rightarrow$ 3 Important Points: It is an NP-Complete problem Used in: Network security // Resource allocation Explain Subset Sum Problem The Subset Sum Problem is a problem where we determine whether there exists a subset of given numbers whose sum is equal to a target value. Problem Statement Given: Set of numbers $S = \{a_1, a_2, \dots, a_n\}$ // Target sum T // Find: A subset whose sum = T Example: // Set: $S = \{2, 4, 6, 10\}$, $T = 16$ // Solution: {6, 10} $\rightarrow$ 16 ✓ Important Points : It is an NP-Complete problem Solved using: // Dynamic Programming // Backtracking	Explain FIFO Branch and Bound FIFO Branch and Bound is a technique where live nodes are stored in a queue (FIFO order) and the first generated node is expanded first. It follows First-In-First-Out principle // Similar to Breadth-First Search (BFS) Key Idea: Nodes are expanded level by level // Always expand the oldest node in queue // Use bounding function to prune nodes Handwritten State Space Tree FIFO Branch and Bound															